Contents

1 Introduction **1**

2 Task Breakdown **1**
2.1 Step 1: Data Peek . . . 1
2.1.1 Design and its motivations . . . 1
2.1.2 Issues . . . 1
2.1.3 Requirements . . . 1
2.1.4 Limitations . . . 1
2.2 Step 2: Data Cleaning . . . 2
2.2.1 Design and its motivations . . . 2
2.2.2 Issues . . . 2
2.2.3 Requirements . . . 2
2.2.4 Limitations . . . 2
2.3 Step 3: Data Visualisation . . . 3
2.3.1 Design and its motivations . . . 3
2.3.2 Issues . . . 3
2.3.3 Requirements . . . 3
2.3.4 Limitations . . . 3

3 Conclusions **4**

4 Division of Work **4**

5 Extra Information **4**
5.1 Future Work Propositions . . . 4

6 Bibliography **5**

1 Introduction

The goal of the assignment was to load, clean, preprocess, analyze, and visualize the given MyAnimeList (MAL) dataset aiming to provide the possibility for fans and journalists to access data about anime.
This repository is organized into two directories:

- **solution**: root directory containing
  - **notebooks**: a collection of Jupyter Notebooks covering the various tasks;
  - **utils**: the modules containig the Python code logic.
- **report**: containing documentation materials;

2 Task Breakdown

2.1 Step 1: Data Peek

2.1.1 Design and its motivations

The design of this task aims to establish a comprehensive overview of the 13 MAL datasets. We implemented a standardized inspection pipeline for each DataFrame to ensure consistency across the exploratory phase:

- **Environment Agnostic Loading**: Centralized path variables (`INPUT_DATA_PATH`) and format aliases (`FILE_EXTENSION`) to allow easy portability.
- **Structural Inspection**: We used `.shape` and `.info()` to identify the volume of entries and the presence of null values; critical for the next phase.
- **Statistical Profiling**: The `.describe()` method was applied to numerical features to detect possible outliers and understand data distribution.
- **Representative Sampling**: Instead of viewing only the first/last few rows, we utilized `.sample(10)` to observe a diverse set of entries and identify potential issues that might have been hidden.

2.1.2 Issues

During the design and implementation of the data peek, several challenges were identified:

- **Memory Consumption**: Loading 13 datasets, including large files like `ratings_df`, required significant RAM usage; as a matter of fact 1 pc out of 3 could't load the dataframe. This issue has been faced via specifc `pandas` loading features per se.
- **Data Type Inconsistencies**: Preliminary inspection revealed that some identifiers, such as `character_mal_id` in the `characters_df`, were loaded as `float64` instead of `int64` due to the presence of NaN values in those columns.
- **Sparsity of Information**: `.info()` highlighted significant missing data in key columns, such as the `about` field in `characters_df` and `season`, `year` fields in the `details_df` dataset.

2.1.3 Requirements

The solution fully complies with the assignment requirements by:

- **Dataset Coverage**: Every provided CSV file in the repository is successfully loaded and inspected.
- **Preliminary Analysis**: The notebook provides a clear "peek" into the contents of each file, documenting the available features and their basic statistics.
- **Data Quality Assessment**: Through the use of `notna()` counts and percentage calculations, the notebook explicitly quantifies the completeness of the datasets.

2.1.4 Limitations

While effective for initial exploration, the current design has limitations:

- **Scalability**: The approach of loading all dataframes into memory at once may fail if the datasets were to increase in size significantly.
- **Lack of Type Casting**: This stage identifies but does not yet correct the identified data type issues, meaning the data is not yet ready for relational joins.
- **Encoding Sensitivity**: Standard encoding assumed; datasets containing Japanese characters (kanji) may require specific parameters if deeper text analysis is performed.

2.2 Step 2: Data Cleaning

2.2.1 Design and its motivations

The design of the cleaning pipeline focuses on maximizing data retention while ensuring the integrity of relational links between datasets; this required us to deepen what only peeked previously so to define a proper cleaning strategy. The implementation is structured as follows:

- **Modular Profiling**: We utilized a custom function, `quick_profile`, to systematically evaluate the dataset for missing values and duplicates before applying any transformations.
- **Selective Row Removal**: Instead of a global `dropna()` approach, we targeted specific *critical* columns to preserve entries that, despite missing secondary information like "descriptions" or "birthdays", still contribute valid relational data.
- **Duplicate Elimination**: Duplicate rows were identified and removed across several datasets so to prevent skewed results in future statistical analyses, ensuring data integrity while maintaining necessary records.
- **Downcasting and Optimization**: To manage the large memory footprint, we also implemented a `downcast_df` utility to optimize data types.

2.2.2 Issues

The implementation phase revealed several data quality challenges:

- **Relational Integrity**: Some rows in the `characters_df` lacked a `character_mal_id`, rendering them useless for joining with other tables; these were removed.
- **Implicit Information**: In the `details_df`, many columns like `season` and `year` had high null counts. We determined these were often valid "missing" values (e.g., movies not having a broadcast season), so the rows were preserved.
- **Non-Standard Nulls**: In `person_details`, we encountered entries where the name was `NaN` but the URL contained strings like "None" or "NULL". Upon verification against the MAL website, these were found to be existing but poorly formatted pages, leading to the decision to keep them.
- **Statistical Representation**: Although `ratings_df` provides individual user ratings, our analysis revealed that it represents only a partial subset of the user base, leading to a significant mean deviation from the official scores. In contrast, `stats_df` contains the full aggregated distribution of ratings, yielding a much lower mean distance. Therefore, to ensure data integrity and statistical accuracy, `ratings_df` was excluded in favor of the more comprehensive `stats_df` for all score-related computations(a specific file, exported as csv, can be generated and found in the `INPUT_DIR_PATH` for further considerations if needed).

2.2.3 Requirements

The cleaning process complies with the assignment requirements by:

- **Data Consistency**: All datasets were processed to remove duplicate entries where they existed.
- **Handling Missingness**: We explicitly handled null values in critical fields that provided no analytical value as for our analysis and visualisation.
- **Exporting Cleaned Assets**: Datframes were exported into a `cleaned/` directory within the `INPUT_DIR_PATH` with a standardized `_clean.csv` suffix to avoid any possible collision during future anlysis.

2.2.4 Limitations

Despite the cleaning efforts, some limitations remain:

- **Heuristic Cleaning**: The decision to keep rows with missing "secondary" info means that future text-mining tasks will still need to handle these nulls on a case-by-case basis.
- **Outlier Preservation**: This stage focused on structural cleaning rather than statistical cleaning.
- **Memory Dependency**: The pipeline still relies on loading DataFrames into memory, which remains a bottleneck unless downcasting is used.

2.3 Step 3: Data Visualisation

2.3.1 Design and its motivations

The solution is designed around a "macro-to-micro" exploration of the given anime ecosystem.
This hierarchical approach is motivated by the need to provide context before detail:

- **Architectural Categories:** The design is organized into four distinct macro categories: **Anime** (content and metadata), **Characters** (on-screen presence), **Creators** (the production engine), and **Users** (audience consumption).
- **Narrative Progression:** We further progress into the **Content and Industry** layers to explore the structural elements that define the medium.
- **Visual Goal:** These choices were made to create a coherent visual narrative that bridges the gap between individual user logs and aggregate engagement stats.

2.3.2 Issues

The design and implementation stages faced challenges regarding technical scale and analytical accuracy:

- **Scalability vs. Interactivity:** The primary challenge was managing the **scale of the dataset**, comprising hundreds of thousands of users and titles, while maintaining smooth interactivity.
- **Data Veracity:** In the interest of academic honesty, it must be noted that we believe the dataset may not be 100% accurate due to potential inconsistencies in user-related metadata. Remaining critical of these underlying quality issues was a central theme in our progress.
- **Defining Success:** Defining "success" in the anime industry was not as-easy-as-expected due to the multi-dimensional nature of the dataset.

2.3.3 Requirements

Efforts have taken place so to answer a simple, yet not so easy, question: WHAT DEFINES SUCCESS ?
We looked for the "secret-recipe" to guide us into the process of possibly making a top-tier anime.
The design fulfills a narrative that transitions from the global landscape to the micro-level mechanics:

- **From Geography to Language:** We track the "geographical story" of anime, mapping its expansion across countries and examining the linguistic diversity of the global audience.
- **The Taxonomy of Success:** To identify the "ingredients" of a hit, we analyzed **genre occurrences** to find dominant thematic clusters. We complemented this with **format affection** analysis, illustrating how delivery methods (TV, Movie, Special) impact user completion rates.
- **User Behavior and Engagement:** We captured the "behavioral story," visualizing the journey from the initial "Plan to Watch" phase to the definitive acts of "Completing" or "Dropping" a series.
- **Behind-the-Scene Components:** At the micro level, we link success metrics to **creators and production roles**, visualizing the "human engine" and staff networks behind high-performing titles.

2.3.4 Limitations

A known limitation is **platform specificity**; the data is derived exclusively from MyAnimeList.
This may not represent the entire global population, particularly in regions where other platforms are dominant. Additionally, while the temporal arc is extensive, the "evolution" is limited by when certain data points began being tracked digitally, potentially under-representing the "pre-digital" era of anime distribution or even incorrect parameters added later on.

3 Conclusions

The design and implementation of this data analysis pipeline provided significant insights into the complexities of handling real-world community-driven datasets like MyAnimeList.

- **Importance of Early Profiling**: The initial "Data Peek" phase was instrumental in identifying structural inconsistencies before they could cause errors in relational joins. This highlighted that data loading and data understanding must be treated as inseparable steps.
- **Trade-offs in Cleaning**: A major lesson learned was the balance between data integrity and data volume. We concluded that a "surgical" approach to null values was superior to a global removal strategy, as it maximized the dataset's utility for diverse types of analysis.
- **Resource Optimization**: The implementation of downcasting utilities was not just an optimization but a requirement to maintain a stable development environment within Jupyter Notebooks.
- **Modularity and Reusability**: Organizing the project into different task-focused modules structures facilitated a more disciplined workflow.

4 Division of Work

Rather than strictly partitioning the tasks into isolated components, our group adopted a collaborative, team-based approach to ensure consistency and knowledge sharing across the entire pipeline.
For each stage we worked collectively on the specific tasks; this resulted in a dynamic workflow where every member participated in: **Development**, **Optimization**, **Refactoring** and **Fixing**.
Our team dynamic was shaped by a mix of different backgrounds.
One of us brought a deep familiarity with the domain. This was a stroke of luck for the cleaning phase; he could instantly tell the difference between a data error and a legitimate empty field.
Complementing this was a member with a strong Python focus, who translated abstract conceptual discussions into functional code, while leading the iterative process of refactoring and performance optimization.
The third member played the vital role of the active listener.
By paying close attention to the friction points between the domain needs as instructed by "the nerd" and the technical constraints denoted by "the coder", ensuring that our different perspectives were unified.
Ultimately, while we didn't start the project with a perfectly defined plan, we managed to navigate the uncertainty and build a robust pipeline through constant, collective refinement.

5 Extra Information

The project was developed using **Visual Studio Code** and is designed to be environment-agnostic.
To facilitate the installation of dependencies, we have provided a `requirements.txt` file in the root directory. This file supports both standard `pip` workflows and `Poetry` users so to set up the environment.
No additional data setup is required beyond ensuring that the MAL datasets are placed in the directory path specified in the `INPUT_DIR_PATH` variable within the notebooks and `graphs_utils.py` module. All custom logic is contained within the `utils/` directory and is automatically imported by the Jupyter Notebooks.

5.1 Future Work Propositions

- **High-Performance Backend**: To overcome the hardware limitations encountered in Step 1, migrating the pipeline to a **Dask-based architecture** would allow for out-of-core computation. This ensures that the massive `ratings_df` dataset can be processed in parallel, making the analysis scalable for future community growth.
- **Interactive Deployment (HCI)**: We envision transitioning from static notebooks to a **Data-as-a-Service (DaaS)** model. By implementing a **Flask-based REST API**, we could power a dynamic frontend "storyboard." This would significantly lower the barrier to entry for journalists and fans, allowing for real-time data exploration through a web interface without the need for a local Python environment.

6 Bibliography

In accordance with the project guidelines, we have primarily relied on the techniques and methodologies presented in the official course slides. Additionally, we incorporated external resources and modern tools to enhance the quality of our data processing and visualization.

- **Generative AI**: tools were utilized to explore advanced plotting techniques and to refine the formatting, ensuring a professional presentation of the results.
- **External References**: For best practices in Exploratory Data Analysis and efficient handling of large-scale datasets, we referenced the methodologies popularized by Rob Mulla[1].

References

[1] Mulla, R. *Data Analysis and Exploratory Data Techniques in Python* `https://github.com/robmulla`.